



Bonus3 实验报告：编译器优化

1. 实现思路

本 Bonus 实现了一套编译器优化管线，在 IR 翻译和代码生成之间插入两个优化阶段：IR 级优化器（`IROptimizer`）和汇编级窥孔优化器（`PeepholeOptimizer`）。同时重构了控制流图构建器，实现真正的基本块划分。

整体优化管线的编译流程如下：

```
IR Translation → IR Optimizer → CFG Builder → Inst Selection  
→ Reg Allocation → Peephole Optimizer → ASM Emitter
```

1.1 IR 级优化器

`IROptimizer` 对每个函数独立应用四轮优化 pass，全部实现于 `src/optimization/ir_optimizer.cpp`（约 350 行）。

1.1.1 常量折叠

维护一个 `std::unordered_map<std::string, int> constants` 跟踪变量到常数值映射。按顺序扫描每条 IR 指令：

- **LoadImm x, #k**：记录 `constants[x] = k`
- **Assign x, y**：若 `y` 为已知常量，折叠为 `LoadImm x, #value`；否则清除 `constants[x]`
- **Unary x, op, y**：若 `y` 为已知常量，在编译期求值 `eval_unary(op, value)`，替换为 `LoadImm x, #result`
- **Binary x, y, op, z**：若 `y` 和 `z` 均为已知常量，编译期求值 `eval_binary(lhs, op, rhs)` 并替换；除法/取模时若除数为零则保留原指令不做折叠（防止 UB）
- **If x, op, y, GOTO label**：若 `x` 和 `y` 均为已知常量且条件成立，替换为无条件 `Goto label`；若条件不成立，不做折叠（保留原指令，后续死代码消除会处理）。折叠后清除常量表——跳转改变了控制流。

常量表在以下位置主动清空：

- 遇到 `Label` 或 `Function` 边界
- 遇到 `Goto`、`Return`、`Call` 等控制流指令
- 遇到 `Store` / `StoreOffset`（可能修改内存中同名变量的值）

1.1.2 死代码消除

`remove_local_dead_defs()` 实现逆向活度分析，从函数末尾反向扫描所有指令：

1. 维护 `live` 集合记录后续会使用的变量名
2. 对纯定义指令（`LoadImm`、`Assign`、`Binary`、`Unary`）：若定义的变量不在 `live` 中，标记为待删除（该定义无任何活跃使用者）；若在 `live` 中，从 `live` 中移除该定义（被当前指令“消费”）
3. 对副作用指令（`Call`、`Store`、`StoreOffset`、`Return`、`Goto`、`If`）：收集其使用的变量名加入 `live`
4. 遇到 `Label` 或 `Function` 时清空 `live` 集合——跨基本块的活度分析需要更完善的数据流框架，这里做保守的局部处理

被标记的指令从最终输出中剔除。

1.1.3 不可达代码消除

`remove_unreachable_after_jumps()` 扫描指令序列，遇到无条件 `Goto` 或 `Return` 后进入“不可达”状态，跳过后续指令直到下一个 `Label` 或 `Function`。

1.1.4 冗余跳转消除

`remove_jump_to_next_label()` 消除紧邻跳转：当 `Goto L` 的下一行恰好就是 `Label L` 时，跳转是冗余的，直接移除。

1.1.5 Pass 编排

`optimize_function()` 的执行顺序：

常量折叠 → 不可达代码消除 → 冗余跳转消除 → 死代码消除 → 冗余跳转消除（再次）

死代码消除后再跑一轮跳转消除，是因为 DCE 可能移除 `If` 后的代码使得紧邻 `Goto → Label` 暴露出来。

1.2 汇编级窥孔优化器

`PeepholeOptimizer` 在寄存器分配完成后对汇编代码做局部优化：

- 自赋值消除： `mv t0, t0`（寄存器到自身的移动）是典型的无操作指令，直接跳过不输出
- 基本块间冗余跳转消除：扫描相邻基本块，若前一个块的最终指令是 `j label_X`，而后一个块的第一条指令正是 `label_X:`，移除该跳转——这是块级版本的"jump-to-next-label"

1.3 基本块划分重构

原有的 `CFGBuilder::build_single_func()` 将整个函数体放在一个基本块内（注释中标注了 `#warning Only one block is created`）。

重构后的实现：

1. **IR 规范化**：将 `Return` 指令展开为 `Assign a0, x + Goto func.ret`（返回值有值）或 `Goto func.ret`（void），这一预处理原在指令选择阶段进行，前移到此便于统一处理
2. **基本块划分**：
 - 遇到 `Label` 指令 → 结束当前块，以该 `label` 为名开始新块
 - 遇到终止指令（`Goto` 或 `If`）→ 该指令结束当前块，自动生成新 `label` 开始下一块
 - 其余指令追加到当前块
3. **控制流边构建**：遍历所有划分后的块，根据最后一条指令建立 `successor/predecessor` 关系：
 - `Goto label` → 添加到目标 `label` 块的边
 - `If ... label` → 添加到目标 `label` 块（真分支）和下一顺序块（假分支，`fall-through`）的两条边
 - 非终止指令结尾 → 添加到下一顺序块的边（`fall-through`）
4. **不可达块剪枝**：从入口块（第一个块）出发做 `BFS`，将所有可达块标记出来，删除不可达块。这一步对后续优化（特别是常量折叠后条件分支变为无条件跳转的场景）至关重要。

1.4 主流程集成

`src/main.cpp` 中 IR 翻译完成后，CFG 构建前插入 `IROptimizer::optimize(ir)`，寄存器分配完成后，汇编发射前插入 `PeepholeOptimizer::optimize(mod)`。

2. 难点与亮点

2.1 难点

常量折叠与副作用指令的交互。Store 和 Call 等指令可能修改变量值，此时常量映射必须保守地清除。最初只在 Store 时清空全部常量表，但过于保守——优化了优化效果。最终选择按 def 精确清除：kill_def 只清除被当前指令定义的变量名，Goto / Return / Call 才全量清空。平衡了安全性与优化效果。

死代码消除的活度分析方向。前向扫描无法知道定义是否有后续使用者——必须先扫描一遍建立活度信息再标记。逆向扫描（从后往前）天然适合这个需求：从使用者反向传播需求量到生产者。但由于不跨越基本块边界（保守地在 Label/Function 处清空 live），局部 DCE 的安全性有保证。

If 常量折叠后的控制流一致性。If 的条件在编译期为真时替换为 Goto，但在假分支的 fall-through 路径上原本还有代码。解决：常量折叠后将假分支代码变为不可达代码，由后续的 remove_unreachable 清理。

Basic block 划分后的出口块处理。划分后的 exit block（func.ret）仍为独立块，但需要确保所有 Return 都跳转到它。在 normalize 阶段将 Return 展开为 Goto func.ret 保证了这一点。

2.2 亮点

- 四轮 Pass 的编排设计：常量折叠 → 不可达消除 → 跳转消除 → DCE → 跳转消除，每轮 pass 都为下一轮创造了更多优化机会，体现了 pass pipeline 的叠加效益
 - 基本块重构：从单块到细粒度多块的划分，配合 BFS 可达性剪枝，为后续的 peephole 跨块优化和寄存分配器的活度分析提供了真正有意义的块边界
 - 窥孔优化的分阶段处理：IR 级和 ASM 级各有侧重——IR 级做语义级优化（常量折叠、DCE），ASM 级做机械式的指令消除（无作用 mv、冗余 j）
 - If 条件常量折叠的分支处理：条件为真时替换为 Goto 并清除常量表（保守处理跳转后的未知状态），条件为假时保留原指令让不可达消除和 DCE 处理
-

3. 心得体会

实现这套优化管线让我对编译器中"优化"的本质有了更立体的认识：

1. 优化的核心不是花哨的算法，而是正确的分析。常量折叠需要准确的常量追踪，DCE 需要准确的活度分析，基本块划分需要正确的终止指令识别。每个优化的前提都是"先理解程序的行为"，然后才是"如何让它更高效"。
2. **Pass** 之间的协同效应远超单个 **Pass** 的效果。常量折叠使条件分支变为无条件跳转 → 暴露不可达代码 → 不可达消除后再暴露冗余跳转 → DCE 移除无用计算 → 再次消除冗余跳转。任一 pass 单独运行效果有限，组合后才产生实质性的代码缩减。
3. 保守性与激进性的权衡无处不在。活跃分析是全局问题但无法跨基本块（需要完整的 CFG + 迭代数据流分析），只能在局部做保守处理（Label 处清空 live）。这意味着某些跨块的死代码会被保留，但绝不会误删活跃的代码——宁可少优化也不要错优化。
4. 基本块划分看似基础，实则深刻影响后续所有阶段。原有的"整个函数一个块"设计使得寄存器分配器只能依赖指令级别的序列号，CFG 层面的优化（如跨块 DCE、公共子表达式消除）完全无法实现。真正的基本块划分为未来的优化扩展铺平了道路。

4. 附加内容

借鉴声明

常量折叠的常量传播/折叠模式参考了编译原理教材中局部值编号和稀疏条件常量传播（SCCP）的基本思想；死代码消除的逆向活度扫描参考了经典的后向活跃变量分析；基本块划分基于教科书标准算法（以 Label 和跳转指令为边界）。未复制他人代码。

AI 工具使用记录

- Proma Agent (Claude Code)：辅助设计 IR 优化器的多 Pass 架构和 evaluate 函数的实现、死代码消除逆向扫描逻辑、CFG 重构中基本块划分和控制流边构建、peephole 优化器实现，以及在 main.cpp 中集成优化管线。不直接生成完整实现代码，由本人独立完成。
- 相关 AI 对话记录保留于仓库 `reports/appends/bonus3/` 目录。